计算机系统基础实验报告

Lab 2：BombLab

姓名：陈璋睿　学号：23300160006　2026 年 10 月 30 日

一、 实验目的

BombLab 给出的是一个随输入变化的验证程序。一次运行中的比较值能够帮助解除当前关卡。但求解函数接收的是任意 `challenge`，实验因而需要恢复数值背后的计算关系。实验文档 [1] 对提交程序的要求也体现了这一点。

我最初面对的困难是怎样解释一次失败。相同的 `BOOM!!!` 可能出现在输入解析阶段，也可能出现在某个中间比较之后。如果只记录最终是否通过，就无法判断原先的理解错在何处。下面的分析围绕各关的主要疑点展开。试验用于区分不同解释，而对反馈的分析决定了后续解法。

二、 实验环境

项目	实际环境
运行环境	Ubuntu 22.04.4 LTS，Linux x86-64 Docker
编译器	GCC 11.4.0，采用 C11
编译选项	`-O2 -Wall -Wextra -Wpedantic -Werror`
调试工具	GDB 12.1
反汇编工具	GNU binutils 2.38
测试脚本环境	Python 3.10.13
个人仓库	`https://github.com/ritachen3656251-design/ICS_lab2`

关卡入口的 `rdi` 指向口令，`rsi` 保存 challenge。不过，这种含义只适用于入口处。执行过程中寄存器会被重新使用，分析一次调用时需要重新确认它的参数。GDB 的反汇编和内存查看方法参考文档 [3, 4]。

本文用 $\oplus$ 表示按位异或。公式中涉及 32 位寄存器的结果均按模 2^{32} 计算，对应代码里的 `uint32_t`。其中 $\mathrm{low}_{32}(x)$ 表示低 32 位，$\mathrm{high}_{32}(x)$ 表示高 32 位。分析时主要使用 seed 42，另用公开列表之外的 seed 43 检查恢复出的关系。

三、 关卡分析

3.1 第 1 关：调用关系

本关的格式检查要求一个整数，随后将这个整数与 `p1_compute` 的返回值比较。输入 0 时程序失败，在该函数返回处可以看到程序确实算出了一个非零的期望值。这个结果能够解释当前输入为什么失败，但还没有解释数值与 challenge 的关系。我因而将分析重点放在返回值的来源上。

`p1_compute` 中出现了三次 helper 调用。起初看函数名，很容易按编号理解它们的先后关系；实际调用却从 step2 开始。更重要的是，下一次调用的参数有一部分来自前一次返回值。如果只分别翻译三个 helper，就会丢失它们之间的数据依赖。

```
call p1_step2
...
mov %eax,%ecx
call p1_step0
...
mov %ecx,%esi
mov %eax,%edi
call p1_step1
```

为弄清这种依赖，我先把 challenge 在调用前形成的三个量记为：

```
low = (uint32_t)challenge;
high = (uint32_t)(challenge >> 32);
middle = (uint32_t)(challenge >> 16);
```

其中 middle 的截取宽度值得单独注意。右移 16 位改变的是位段起点。随后写入 32 位寄存器的操作决定了保留宽度，因此 middle 包含原数的第 16 至 47 位。把它理解成一个 16 位字段，会丢掉仍参与计算的高半部分。

再把每次调用的参数与 helper 内部运算对应，可以恢复出：

```
a = (5u * high) ^ (low + 0xb15eu);
b = (5u * low + middle) ^ 0x1db3u;
answer = (b + 4u * a) ^ 0xe881u;
```

这样，观察到的返回值就成为公式的一个验证样本。求解函数对新的 challenge 重新形成三个量，再计算 a 和 b。seed 43 的运行通过了原程序检查，结果见图1。

本关使我认识到，理解函数调用需要追踪值在调用之间怎样传递。helper 的局部计算与调用者组织这些计算的方式共同决定最终结果。

```
Phase 1 | seed 43

$ ./defuser --seed 43 --phase 1
2034676901

$ ./bomb --seed 43 --phase 1
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
```

图 1: 第 1 关在 seed 43 下的验证结果

3.2 第 2 关：递推关系

本关读取六个整数，随后逐项比较。输入六个零后，首项比较处的输入值仍为 0。程序在同一位置算出的期望值则是 14。这一反馈把问题缩小到了首项的生成方式，后五项暂时还没有参与当前失败。

首项比较前的运算很短：

```
shr $0xa,%rax
and $0x1f,%eax
add $0x7,%eax
```

由此得到 $v_0 = ((c \gg 10)\ \&\ 31) + 7$。右移确定所取位段的位置，掩码保留其中五位。这个解释在 seed 42 下给出 14，在 seed 43 下给出 29。补对首项使程序进入了循环。新的失败发生在第二项比较。于是可以把首项关系和后续递推分开研究。

循环第一轮的主要指令为：

```
lea 0x7(%rdx),%ecx
shr %cl,%rax
and $0xf,%eax
add %edx,%eax
xor %esi,%eax
mov -0x40(%rbp,%rdx,4),%esi
lea (%rax,%rax,2),%eax
cmp %eax,%esi
```

第一轮下标为 1，移位量因而是 8。seed 42 中的计算对应于 $3((15+1) \oplus 14) = 90$。

这里存在一个会误导数值推断的巧合：16 与 14 的二进制表示没有重叠的 1，因此二者相加和异或恰好同为 30。如果只观察这一轮的结果，“先相加再乘 3” 也能解释 90。我把加法关系写成一个对照版本，用 seed 43 检验它。

新的前项是 29，片段加上下标后为 9。9 与 29 存在重叠的 1。加法会在这些位置产生进位，异或则会抵消对应位。对照版本因而算出第二项 114，与原程序期望的 60 出现差别。

seed	前项	片段加下标	异或后乘 3	加法后乘 3
42	14	16	90	90
43	29	9	60	114

输入第二项 114 时，失败发生在这一项的比较；改成 60 后，程序进入下一轮。这个结果排除了加法解释。这个反例体现了换输入的作用：让原本难以区分的解释产生不同预测。

运算符确定以后，还需要解释 `esi` 在循环中的含义。执行 `xor` 时，它保存已经比较通过的前项；随后的 `mov` 才将当前输入读入同一个寄存器。当本轮通过并进入下一轮时，当前项自然成为新的前项。

我用 seed 42 的第三项继续核对。第二轮中的前项已经是 90。该轮移位量变成 9，取出的片段仍是 15。于是

$$(15 + 2) \oplus 90 = 75, \qquad 3 \times 75 = 225.$$

这与原程序对第三项的期望一致，支持“每轮依赖紧邻前项”的判断。

最终关系为

$$v_0 = ((c \gg 10)\ \&\ 31) + 7, \qquad v_i = 3\big(v_{i-1} \oplus (p_i + i)\big), \quad p_i = (c \gg (i + 7))\ \&\ 15.$$

下标从 1 到 5，恰好生成首项以外的五项。相邻轮次的移位量只增加一位，因此取出的位段会重叠。代码按这个关系直接递推，seed 43 的完整结果见图2。本关的主要收获是把“某个值对得上”与“对产生这个值的机制理解正确”区分开来。

```
Phase 2 | seed 43

$ ./defuser --seed 43 --phase 2
29 60 150 477 1398 4212

$ ./bomb --seed 43 --phase 2
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.
```

图 2: 第 2 关在 seed 43 下的验证结果

3.3 第 3 关：跳转表

本关有两项输入。第一项与 challenge 的低三位比较，因而可以先恢复出索引。第二项来自 dispatcher。函数名中的 case 编号能否直接对应输入索引，仍需要进一步判断。

dispatcher 中真正决定分支的是间接跳转：

```
cmp $0x7,%eax
ja  <out_of_range>
notrack jmp *0x405820(,%rax,8)
```

比例 8 表示每个表项保存一个 64 位目标地址。读取表项并跟到目标后，发现索引 0 实际进入 case6。这使我重新区分了函数标签和选择关系：标签帮助识别代码块，输入索引则通过表项决定执行哪一块。

为了单独检验映射，我选择 challenge 0。此时参与分支计算的参数也为 0，运算中的变量影响被消去。如果按“索引 0 对应 case0”计算，会得到第二项 19859；实际试验失败。按表项选择 case6 后得到 34764，同一输入条件下运行通过。这个对照将失败原因指向了分支映射，参数计算则在两次试验中保持相同。

令

$$i = c\ \&\ 7, \qquad x = \mathrm{low}_{32}(c \gg 3) \oplus \mathrm{high}_{32}(c),$$

各分支恢复为下表。表中的常量来自相应 helper 的立即数。

索引	目标	第二项
0	case6	`(x ^ 0x87ccu) + (x >> 2)`
1	case7	`5u*x + 0x49160u`
2	case2	`5u*x + 0x9a24u`
3	case0	`(x ^ 0x4d93u) + (x >> 2)`
4	case1	`5u*x + 0xb829u`
5	case5	`3u*x + 0xfe6au`
6	case3	`(4u*x) ^ 0x833du`
7	case4	`(2u*x) ^ 0x1105u`

实现中的 switch 按索引组织，每一项使用表中确定的 helper 关系。额外测试分别覆盖八种索引；seed 43 的验证结果见图3。本关中的跳转依据提供了分支选择关系。恢复这层关系以后，各代码块的计算才能与输入索引正确对应。

Phase 3 | seed 43

```
$ ./defuser --seed 43 --phase 3
7 3149956987

$ ./bomb --seed 43 --phase 3
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
```

图 3: 第 3 关在 seed 43 下的验证结果

3.4 第 4 关：递归返回

从 `p4_walk` 的区间参数和中点比较，可以判断它具有二分查找的结构。初步的疑问是：返回值是否就是找到的目标，还是还记录了搜索路径的信息？长路径会把两种影响混在一起。我因而先构造立即命中中点的情况。

目标为 challenge 的低七位，初始区间是 0 至 127。challenge 63 的目标等于首次中点，其 salt 参数也恰好为 0。输入目标值 63 仍然失败，而函数的返回值是 311。基本分支的指令解释了两者的差别：

```
lea 0xf8(%rdi),%eax
xor %ecx,%eax
ret
```

它在中点上加 248，再与 salt 异或。因此，即使命中目标也要形成一个新的值。将输入改为 311 后，最短路径的试验通过。这一反馈支持了对基本情况的解释。

接下来需要判断上层是否直接传递子调用结果。我取 challenge 31，使搜索只经过一层左分支。子调用在区间 $[0, 62]$ 命中 31 并返回 279。但回到外层以后，整个函数的返回值变成了 886。这个差别说明，求解时还要计算回溯阶段。

父调用在递归前保存了当前中点与 salt 的和。`lea` 在递归返回后将子调用结果乘 3。随后两次异或分别加入了保存值和左分支常量 12。本次试验对应的计算是 $837 \oplus 63 \oplus 12 = 886$，其中 $837 = 3 \times 279$。右分支具有相同结构，最后的常量改为 10。

令中点 $m = l + \lfloor (h-l)/2 \rfloor$，递归关系为

$$F(t,l,h,s) = \begin{cases} (m+248) \oplus s, & t = m, \\ 3F(t,l,m-1,s) \oplus (m+s) \oplus 12, & t < m, \\ 3F(t,m+1,h,s) \oplus (m+s) \oplus 10, & t > m. \end{cases}$$

其中

$$t = c \,\&\, 127, \qquad s = \mathrm{low}_{32}(c \gg 7) \oplus \mathrm{high}_{32}(c).$$

这一关系解释了为什么最终结果与整条搜索路径有关。基本分支给出最内层结果，每一层返回时又加入该层中点的信息。代码因而保留递归写法，便于对应这种依赖。

测试覆盖了全部 128 种目标，seed 43 的结果见图4。本关让我把递归区分成进入下一层和从下一层返回两个方向；只理解区间怎样缩小，还不足以恢复整个函数。

```
Phase 4 | seed 43

$ ./defuser --seed 43 --phase 4
3949797497

$ ./bomb --seed 43 --phase 4
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
```

图 4: 第 4 关在 seed 43 下的验证结果

3.5 第 5 关：链表遍历

本关检查两项输入。遍历函数通过返回值给出第一项，通过指针参数写回第二项。这使我首先关注：除了某种累计计算，遍历是否还需要留下末尾节点的信息?

函数从起点表取出一个指针，随后访问节点内的固定偏移：

```
mov 0x405b60(,%rax,8),%rdx
mov 0x8(%rdx),%esi
add 0xc(%rdx),%eax
...
mov (%rdx),%rdx
```

首个字段随后作为新地址使用，因而可以解释为 next 指针。另两个字段参与算术或被写回输出，宽度都是 4 字节。由此恢复出一个 16 字节节点：前 8 字节保存 next，后面依次是 id 和 bias。

我最初将 `x/8uw` 的输出理解成八个指针。实际上 w 的单位是 4 字节，读到的零只是若干指针的高半部分。改用 8 字节单位后，才读全八个起点。这个错误的原因在于把“显示几个单元”和“显示几个对象”混为一谈。

继续跟随 next 后，看到相邻处理的节点并不按数组下标递增。例如第一个起点落在 nodes[5]，它的 next 指向 nodes[8]。求解程序因而需要保留静态节点之间的链接关系。遍历顺序由 next 指针决定。

处理次数还有一个疑点：从 challenge 得到的两位字段应当怎样解释？它可能表示总处理数，也可能只表示额外前进的次数。我先取 challenge 0，它给出的该字段为 0。直接把这个字段当作循环次数会跳过全部节点。但实际函数已经完成一次累计更新，写回的 id 为 1049。这说明首节点在循环之前就参与了计算。

为了进一步区分两种解释，我把 challenge 改为 8。这次低三位保持不变，起点因而相同。用于累计初值的移位结果也相同。发生变化的只有控制长度的字段。这样，输出变化就可以归因于处理长度。

沿用原来的答案会失败。单步跟随 next 后，看到新节点的 id=55、bias=37381。将新节点纳入第二次更新后，累计值变成 322556。写回的 id 同时变成了 55。新的两项答案通过了检查。这一反馈同时确定了长度的含义和第二项输出的含义：它保存最后处理节点的 id。

累计初值为

$$v = \mathrm{low}_{32}(c \gg 6) \oplus \mathrm{high}_{32}(c) \oplus 0\mathrm{x}7581.$$

起点由低三位选择。令 $k = (c \gg 3)\,\&\,3$，则总共处理 $k+1$ 个节点。每个节点的更新为

$$v \leftarrow (3v + \mathrm{bias}) \oplus \mathrm{id}.$$

代码用一个循环统一包含首节点，每轮同时更新末尾 id。测试分别组合八个起点和四种长度，seed 43 的结果见图5。

本关中，长度字段的值只有结合循环前的操作才有完整含义。若只盯着循环内部，很容易把首节点漏掉。选用起点和初值都相同的对照输入，则使这个边界问题变得清楚。

```
Phase 5 | seed 43

$ ./defuser --seed 43 --phase 5
3601218346 1313

$ ./bomb --seed 43 --phase 5
Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.
```

图 5: 第 5 关在 seed 43 下的验证结果

3.6 第 6 关：间接调用

本关入口要求一个整数字段。此前沿用双整数占位输入会在进入 dispatcher 以前失败。纠正数量后，才能观察实际计算。dispatcher 有两次间接调用。第一步需要确定它们各自的目标，随后才能分析第一次返回值如何成为第二次的参数。

函数表具有八项，但其中只有六个不同的目标。这说明选择值的范围与不同函数的数量并不相同。恢复出的表为：

索引	0	1	2	3	4	5	6	7
目标	op1	op5	op4	op0	op4	op3	op2	op3

有一个目标只对参数执行 `ror`。challenge 67 会选中这个函数，并使第一次调用的参数为 1。若只将它理解成普通右移，结果应为 0；实际返回的是 2^{31}。这说明原最低位被移到了最高位，必须按 32 位循环右移解释。

这个样本把“移出位被丢弃”和“移出位被补回”两种理解分开。其余目标也按各自指令恢复为：

函数	32 位计算关系
op0	`(x >> 1) | (x << 31)`
op1	`(2u*x + 0xc3ecu) ^ (7u*x)`
op2	`(x ^ (x >> 7)) + 0xe4a9u`
op3	`(x + 0xca25u) ^ (x >> 2)`
op4	`((x << 2) | (x >> 30)) ^ 0x62e5u`
op5	`(x ^ 0xf6bcu) + (x << 7)`

challenge 0 的第一次调用返回 50156。直接将它作为答案，程序仍然失败。调用者会先将第一次结果与常量异或。这个新值成为第二函数的参数，其返回值还参与了末尾加法。

```
call *0x405e00(,%rax,8)
xor $0x8322,%eax
mov %eax,%edi
call *0x405e00(%rbx)
add $0x671c,%eax
```

同一样本中第二函数返回 32810。末尾加法进一步得到最终结果 59206。由此，失败的原因可以解释为只完成了组合计算的第一部分。

设初值 $x = \mathrm{low}_{32}(c \gg 6) \oplus \mathrm{high}_{32}(c)$，两个索引分别为 $i = c \,\&\, 7$ 和 $j = (c \gg 3) \,\&\, 7$，完整关系为

$$F(c) = O_j\big(O_i(x) \oplus 0\mathrm{x}8322\big) + 0\mathrm{x}671\mathrm{c}.$$

这里 O_i 表示表中第 i 项选中的操作。实现保留函数指针表，使两次调用的关系在代码中直接可见。

额外测试覆盖了两次索引的全部组合，seed 43 的结果见图6。本关的关键是同时理解被调用函数和调用者：局部操作确定怎样算，调用者决定这些操作怎样连接。

Phase 6 | seed 43

```
$ ./defuser --seed 43 --phase 6
591440785

$ ./bomb --seed 43 --phase 6
Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.
```

图 6: 第 6 关在 seed 43 下的验证结果

四、 Honor 分析

4.1 输入布局

六关通过后，程序继续读取第七行。输入 `test` 能到达 `secret_gate`，但很快返回失败。此时若直接猜数字关系，就会研究一个尚未真正执行的条件。因此，我先判断原输入经过了哪些处理。

长度检查对 `strlen-36` 作无符号比较，对应可接受的长度区间为 36 至 163。后面的复制和固定位置比较表明，输入中还包含一个标志字段。比较位置距离复制起点 32 字节，其后 4 字节才是交给 `secret_phase` 的子串：

```
lea -0xd0(%rbp),%r12
...
cmpl $0x4e45504f,-0xb0(%rbp)
lea -0xac(%rbp),%rdi
call secret_phase
```

把比较常量按小端字节解释，得到 `OPEN`。这使我能够按后续访问的用途确定各字段的位置。

为了核对位置，我对比了 31 个和 32 个填充字符。前一种输入在标志处读到错位的 `PEN0`，未进入数字检查。后一种输入能进入 `secret_phase`；此时函数看到的内容恰好是 `0 0 0`。32 字节填充把标志放到了指定字段。其后的数字文本因而能够交给下一层函数处理。

4.2 数值关系

数字后缀包含三个整数。第一个整数只与 context 的低两位比较；另外两个整数同时依赖 challenge 和 context。由于 context 来自本次六关的口令，恢复公式时需要把它作为独立输入看待。

seed 42 中的第一项期望为 0，初始的三个零已经通过这一项。程序随后在第二项失败，说明字符串布局与第一项条件已经满足。继续观察第二项的计算，得到下面的关系：

```
mode = (uint32_t)(context & 3u);
a = (uint32_t)challenge ^ (uint32_t)(challenge >> 32);
b = (uint32_t)context ^ (uint32_t)(context >> 32);
x = a ^ b;
x = (x >> 5) | (x << 27);
first = x ^ 0xf159u ^ (uint32_t)(context >> 16)
        ^ (mode * 0x6a9eu);
second = first ^ (b + mode + 0x9bc7u + (a >> 5));
```

这里 a 和 b 分别折叠了两个 64 位输入的高低半部。`secret_mix` 对二者异或后的值循环右移五位，再与常量异或。调用者又加入 context 中间位段的影响，形成第二项 first。

我用恢复出的关系补上 first，同时保持最后一项为零。这次第二项比较通过，失败位置移到了第三项。因而新的失败提供了两层信息：前面的计算已经得到原程序认可，剩余问题位于 first 返回之后。继续解释末尾求和与异或，得到 second 的关系。补齐后，seed 42 的完整运行通过 Honor。

在这一步中，`context>>16` 的宽度尤其重要。右移以后仍会截取 32 位，因此使用的是原数的第 16 至 47 位。若将它只理解为高 16 位，第二项就会遗漏输入的一部分影响。

seed 42 的 mode 恰好为 0，这意味着该样本中 mode 乘常量的部分也为 0。漏写这一部分也可能得到相同结果。完成这个样本后，仍需用不同 mode 的输入检查公式。

seed 43 的 mode 变成 1，其运行通过结果见图7。进一步使用的 20 个列表外 seed 覆盖了四种 mode，完整输出均通过原程序检查。这样的验证针对的是公式中尚未在初始样本中发挥作用的分量。

Honor 的分析也使我重新理解失败的含义。入口处失败只能反映长度或标志条件；数字检查处失败才与后缀公式有关。当失败位置随着局部修正向后移动时，这种移动本身就能为已经恢复的部分提供证据。

Honor | seed 43

```
$ { ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
```

图 7: seed 43 下六关与 Honor 的完整验证结果

五、 验证与复盘

最终求解程序通过六关公开测试，共计 60/60；Honor 公开测试为 10/10。每关还以公开列表之外的 seed 43 单独验证，运行结果已放在相应分析之后。额外测试所针对的疑点见下表。

关卡	公开测试	额外测试针对的问题
1	10/10	高半部变化后，helper 参数关系是否仍成立
2	10/10	首项取值改变后，递推解释是否仍与原程序一致
3	10/10	每一种索引是否都对应正确分支
4	10/10	全部目标位置能否按恢复的返回关系计算
5	10/10	起点与处理长度改变后，末尾 id 是否随遍历更新
6	10/10	两次选择值组合后，函数连接关系是否仍成立
Honor	10/10	mode 改变时，context 相关的计算是否仍正确

```
Public tests | captured summary

$ python3 public_test.py
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
[exit status: 0]
```

图 8: 公开测试结果

第二关最能体现这次实验中的认识变化。加法解释和异或解释对 seed 42 具有相同预测。单次通过因而无法帮助我作出选择。seed 43 中重叠的二进制位使两个结果不同，错误解释由此暴露出来。这使我在选择测试输入时开始关注它能回答的问题。一个有作用的试验应当使不同解释产生能够观察到的差别。

第四关则让我注意到，观察到的值还需要放回调用层次中解释。子调用的 279 和整体函数的 886 都是真实结果，但它们属于不同调用层次。若把前者直接当成答案，就会漏掉回溯阶段表达的关系。这种区别也适用于第六关中两次间接调用的返回值。

本次求解最终形成了与每关检查条件对应的计算关系。局部修正是否消除了原先的矛盾，需要通过反馈判断。改变输入条件以后继续检验解释，才能逐渐确定它的适用范围。报告选取这些反馈，是为了说明公式为什么能够从程序中恢复出来。

六、 参考资料

[1] 课程实验文档：Lab2 BombLab。https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/。

[2] 实验仓库的 README.md。教学目录中的说明用于对照调用约定与观察方法。

[3] GNU GDB Manual：Source and Machine Code。
https://sourceware.org/gdb/current/onlinedocs/gdb.html/Machine-Code.html。

[4] GNU GDB Manual：Examining Memory。
https://sourceware.org/gdb/current/onlinedocs/gdb.html/Memory.html。

七、 AI 使用说明

1. 用于拆解题目并分析题目条件。
2. 用于解释相关概念，并辅助理解题设。
3. 针对具体报错分析原因，并解释调试过程中涉及的代码。
4. 用于编译报告 PDF，并修改报告语言。